

Pluribus for Codex

四人 **Codex** 开发指令

Memory Registry Pivot · 独立上下文重置 · 分支边界 · 验收标准

使用方式：将对应章节中的完整英文指令直接发送给该成员的 **Codex**。

四个开发分支必须基于同一基线 **Commit : 1b5e078d70b807ad92453737882f797eae6cc423**

分工总览

成员	开发分支	唯一职责
Person 1	feat/memory-registry-api	Memory Registry API 与持久化
Person 2	feat/codex-memory-runtime	本地安装、匹配与 Codex 上下文注入
Person 3	feat/memory-usage-verification	Usage Receipt、演示夹具与验证
Person 4	feat/memory-marketplace-ui	Memory Marketplace UI 与演示路径

统一产品闭环：**Publish** → **Discover** → **Install** → **Codex Uses** → **Verify**

注意：旧的多 **Agent Mission Control** 方向已经废弃。每个 **Codex** 必须先读取 **AGENTS.md** 和两份新范围文档，再开始编码。

1. Person 1 — Memory Registry API

以下内容请完整复制给该成员的 **Codex** :

You are Person 1 working on Pluribus for Codex.

MANDATORY CONTEXT RESET:

Your previous memory may describe this repository as a multi-agent Mission Control system using Scout, Builder, Tester, Reviewer, Hive Blackboard, Git worktrees, and one shared coding mission.

That product direction is obsolete. Do not continue it.

The current product is:

“GitHub for useful Codex memories.”

Pluribus allows developers to publish, discover, install, reuse, fork, and verify Codex debugging methods and coding best practices.

The current product loop is:

Publish → Discover → Install → Codex Uses → Verify

YOUR BRANCH:

feat/memory-registry-api

EXPECTED BASE COMMIT:

1b5e078d70b807ad92453737882f797eae6cc423

Before writing code:

1. Run:
 - git branch --show-current
 - git rev-parse HEAD
 - git status --short
2. Confirm the branch is feat/memory-registry-api.
3. Confirm AGENTS.md exists.
4. Read completely:
 - AGENTS.md
 - docs/MEMORY_REGISTRY_SCOPE.md
 - docs/TEAM_EXECUTION_PLAN_V2.md
5. Summarize the new product definition, your directory ownership, the frozen API contract, prohibited changes, and acceptance tests.

Do not write code until you have confirmed the context reset.

YOUR ROLE:

You own the Memory Registry API and persistence layer.

YOU MAY MODIFY ONLY:

```

backend/app/registry/**
backend/app/api/memories.py
backend/app/schemas/memories.py
backend/app/db/database.py
backend/app/main.py
backend/tests/registry/**
backend/tests/api/test_memories.py

```

Only you may register the Memory Registry router in app/main.py or change the registry tables in database.py.

YOUR TASKS:

1. Write failing tests for the frozen Memory Capsule schema.
2. Implement typed models for Author, Memory verification, Memory Capsule, Install record, and Demo Snapshot.
3. Add idempotent SQLite tables for memories, memory_stars, and memory_installs.
4. Seed this fixed verified memory exactly once:
 - ID: mem_pytest_importlib_v1
 - Slug: fix-pytest-module-collisions
 - Title: Fix duplicate pytest module collisions
5. Implement search across title, summary, triggers, and tags.
6. Implement the frozen endpoints:
 - GET /api/memories?query=&tag=&sort=
 - POST /api/memories
 - GET /api/memories/{slug}
 - POST /api/memories/{slug}/star
 - POST /api/memories/{slug}/install
 - GET /api/installed
 - GET /api/demo/snapshot
7. Preserve the existing /health endpoint, localhost-only CORS, existing database tables, and all existing passing tests.

REQUIRED OBSERVABLE RESULT:

GET /api/memories?query=pytest must return the seeded verified memory.

GET /api/demo/snapshot must return the exact aggregate shape from docs/MEMORY_REGISTRY_SCOPE.md.

DO NOT:

- Do not build the local filesystem installer.
- Do not run Codex.
- Do not modify frontend/.
- Do not add authentication, billing, or social following.
- Do not modify the frozen response fields.
- Do not continue the old Mission Control product.

DEVELOPMENT METHOD:

Follow strict RED → GREEN → REFACTOR. For each behavior: write the failing test; run it and confirm the expected failure; add the smallest implementation; run the focused test; run the full suite; commit a small coherent change.

REQUIRED VERIFICATION:

```
cd backend
uv sync --dev
uv run pytest tests/registry tests/api/test_memories.py -q
uv run pytest -q
uv run ruff check .
uv run ruff format --check .
```

DELIVERY:

Push only to feat/memory-registry-api.

Open a Draft PR targeting pivot/memory-registry-v1. The PR description must include real test output, changed files, implemented endpoints, and remaining blockers.

Begin by reading the required documents and reporting your understanding. Do not begin implementation until the branch and product direction are confirmed.

2. Person 2 — Codex Memory Runtime

以下内容请完整复制给该成员的 **Codex** :

You are Person 2 working on Pluribus for Codex.

MANDATORY CONTEXT RESET:

Your previous memory may describe this repository as a multi-agent Mission Control system using Scout, Builder, Tester, Reviewer, Hive Blackboard, Git worktrees, and one shared coding mission.

That product direction is obsolete. Do not continue it.

The current product is:

“GitHub for useful Codex memories.”

Pluribus allows one developer to publish a useful Codex debugging method and another developer to install and reuse it.

The current product loop is:

Publish → Discover → Install → Codex Uses → Verify

Pluribus does not mutate hidden Codex model memory. It manages explicit local memory packages and injects matching installed memories into bounded Codex task context.

YOUR BRANCH:

feat/codex-memory-runtime

EXPECTED BASE COMMIT:

1b5e078d70b807ad92453737882f797eae6cc423

Before writing code:

1. Run:


```
git branch --show-current
git rev-parse HEAD
git status --short
```
2. Confirm the branch is feat/codex-memory-runtime.
3. Confirm AGENTS.md exists.
4. Read completely:


```
AGENTS.md
docs/MEMORY_REGISTRY_SCOPE.md
docs/TEAM_EXECUTION_PLAN_V2.md
```
5. Summarize the new product definition, your directory ownership, how local memory installation works, the honest Codex boundary, prohibited changes, and acceptance tests.

Do not write code until you have confirmed the context reset.

YOUR ROLE:

You own the local Memory Runtime, installation format, trigger matching, redaction, and Codex context injection.

YOU MAY MODIFY ONLY:

```
backend/app/memory_runtime/**
backend/app/adapters/codex/memory_context.py
backend/tests/memory_runtime/**
tools/pluribus_memory.py
```

YOUR TASKS:

1. Write failing tests for redacting fake bearer tokens, fake API-key assignments, and personal absolute home-directory paths.
2. Render a deterministic MEMORY.md from the frozen Memory Capsule.
3. MEMORY.md must include title, memory ID, version, author, problem, triggers, steps, compatibility, and verification evidence.
4. Generate manifest.json containing memory_id, slug, version, install_path, content_sha256, and installed_at.
5. Install memories under <root>/.pluribus/installed/<slug>/.
6. Installation must create MEMORY.md and manifest.json.
7. Reinstalling the same version must be idempotent.
8. Implement list installed memories, uninstall memory, trigger matching, and bounded context-packet generation.
9. Match task or error text against triggers, tags, and problem description.
10. Add a helper that injects the selected memory packet into the existing Codex adapter prompt.
11. Provide CLI commands:

```
python tools/pluribus_memory.py install --capsule <memory.json> --root <repository>
python tools/pluribus_memory.py list --root <repository>
python tools/pluribus_memory.py match --text "<error text>" --root <repository>
```

FIXED MEMORY:

ID: mem_pytest_importlib_v1
 Slug: fix-pytest-module-collisions
 Trigger: pytest import file mismatch caused by duplicate test module basenames
 Method: configure pytest with --import-mode=importlib

REQUIRED OBSERVABLE RESULT:

After installation, this command:

```
python tools/pluribus_memory.py match --text "pytest import file mismatch in duplicate test_runner.py" --root /tmp/pluribus-demo
```

must select mem_pytest_importlib_v1.

The generated context packet must explicitly contain the memory ID, memory version, matched trigger, and reusable steps.

DO NOT:

- Do not create Registry API routes.
- Do not change Registry database tables.
- Do not edit backend/app/main.py or frontend/.
- Do not claim hidden Codex memory mutation.
- Do not read or commit ~/.codex/auth.json.
- Do not commit raw private Codex transcripts, real secrets, or personal home paths.
- Do not continue the old Mission Control product.

DEVELOPMENT METHOD:

Follow strict RED → GREEN → REFACTOR. For each behavior: write the failing test; run it and confirm the expected failure; add the smallest implementation; run the focused test; run the full suite; commit a small coherent change.

REQUIRED VERIFICATION:

```
cd backend
uv run pytest tests/memory_runtime -q
uv run pytest -q
uv run ruff check .
uv run ruff format --check .
```

Also manually verify the install, list, and match CLI commands using a temporary directory and fake data only.

DELIVERY:

Push only to feat/codex-memory-runtime.

Open a Draft PR targeting pivot/memory-registry-v1. The PR description must include real test output, CLI output, installed file paths under the temporary root, and remaining blockers.

Begin by reading the required documents and reporting your understanding. Do not begin implementation until the branch and product direction are confirmed.

3. Person 3 — Usage Receipt 与验证

以下内容请完整复制给该成员的 **Codex** :

You are Person 3 working on Pluribus for Codex.

MANDATORY CONTEXT RESET:

Your previous memory may describe this repository as a multi-agent Mission Control system using Scout, Builder, Tester, Reviewer, Hive Blackboard, Git worktrees, and one shared coding mission.

That product direction is obsolete. Do not continue it.

The current product is:

“GitHub for useful Codex memories.”

The product must prove that a debugging method published by one developer was installed by another developer, injected into Codex, used on a matching problem, connected to a concrete file change, and followed by successful coordinator-run verification.

The current product loop is:

Publish → Discover → Install → Codex Uses → Verify

YOUR BRANCH:

feat/memory-usage-verification

EXPECTED BASE COMMIT:

1b5e078d70b807ad92453737882f797eae6cc423

Before writing code:

1. Run:

```
git branch --show-current
git rev-parse HEAD
git status --short
```

2. Confirm the branch is feat/memory-usage-verification.

3. Confirm AGENTS.md exists.

4. Read completely:

```
AGENTS.md
docs/MEMORY_REGISTRY_SCOPE.md
docs/TEAM_EXECUTION_PLAN_V2.md
```

5. Summarize the new product definition, your directory ownership, the fixed demo failure, the Usage Receipt contract, the live/replay boundary, prohibited changes, and acceptance tests.

Do not write code until you have confirmed the context reset.

YOUR ROLE:

You own the fixed failing repository fixture, Usage Receipt generation, coordinator verification, and sanitized live/replay evidence.

YOU MAY MODIFY ONLY:

```

backend/app/usage/**
backend/tests/usage/**
demo/memory-fixture/**
demo/memory-evidence/**
demo/run-memory-demo.py

```

YOUR TASKS:

1. Create a minimal Python fixture with duplicate test module basenames.
2. The baseline fixture must reproducibly produce a pytest import-file-mismatch collection failure without importlib mode.
3. Preserve a sanitized failing-baseline artifact under demo/memory-evidence/.
4. Write failing tests for the frozen Usage Receipt contract.
5. Implement receipt construction from memory match, installed memory ID, matched trigger, injected Codex context, Codex-reported use, changed files, and coordinator-run verification.
6. Reject receipts missing any required proof.
7. Reuse the existing safe subprocess verification runner.
8. Never use shell=True.
9. Create demo/run-memory-demo.py.
10. The runner must support explicit modes: --mode live and --mode replay.
11. Replay must always display or emit: REPLAY — RECORDED EVIDENCE.
12. The final demo must use:
 - Memory ID: mem_pytest_importlib_v1
 - Matched trigger: import file mismatch
 - Expected changed file: pyproject.toml
 - Verification command: pytest -q
 - Expected exit code: 0
13. Generate sanitized replay evidence that Person 4 can consume without inventing fields outside the frozen contract.

REQUIRED OBSERVABLE RESULT:

The final proof chain must be:

```

baseline pytest failure
→ mem_pytest_importlib_v1 selected
→ installed memory injected into Codex context
→ Codex reports using the memory
→ pyproject.toml changed
→ coordinator runs pytest -q
→ exit code 0
→ Usage Receipt generated

```

The Usage Receipt must connect publisher, memory ID and version, consumer, matched trigger, Codex injection, Codex-reported use, changed files, verification command, exit code, and sanitized output excerpt.

DO NOT:

- Do not add Memory Registry API routes.
- Do not modify backend/app/main.py or backend/app/db/database.py.
- Do not edit frontend/.
- Do not fabricate live Codex output.

- Do not present Replay as Live.
- Do not commit secrets or absolute personal paths.
- Do not continue the old Mission Control product.

DEVELOPMENT METHOD:

Follow strict RED → GREEN → REFACTOR. For each behavior: write the failing test; run it and confirm the expected failure; add the smallest implementation; run the focused test; run the full suite; commit a small coherent change.

REQUIRED VERIFICATION:

```
cd backend
uv run pytest tests/usage -q
uv run pytest -q
uv run ruff check .
uv run ruff format --check .
```

Then run the final fixture verification from its own directory and save only sanitized evidence.

DELIVERY:

Push only to feat/memory-usage-verification.

Open a Draft PR targeting pivot/memory-registry-v1. The PR description must include the real baseline failure signature, final verification output, generated Usage Receipt path, changed file list, live/replay status, and remaining blockers.

Begin by reading the required documents and reporting your understanding. Do not begin implementation until the branch and product direction are confirmed.

4. Person 4 — Memory Marketplace UI

以下内容请完整复制给该成员的 **Codex** :

You are Person 4 working on Pluribus for Codex.

MANDATORY CONTEXT RESET:

Your previous memory may describe this repository as a multi-agent Mission Control UI with Scout, Builder, Tester, Reviewer, Agent Roster, Hive Memory, Coordinator Evidence, and one shared coding mission.

That product direction and UI are obsolete. Do not continue them.

Do not reuse the old Mission Control narrative as the primary product.

The current product is:

“GitHub for useful Codex memories.”

The UI should feel like GitHub, a package registry, or a marketplace for reusable Codex debugging methods and coding best practices.

The current product loop is:

Publish → Discover → Install → Codex Uses → Verify

YOUR BRANCH:

feat/memory-marketplace-ui

EXPECTED BASE COMMIT:

1b5e078d70b807ad92453737882f797eae6cc423

Before writing code:

1. Run:


```
git branch --show-current
git rev-parse HEAD
git status --short
```
2. Confirm the branch is feat/memory-marketplace-ui.
3. Confirm AGENTS.md exists.
4. Read completely:


```
AGENTS.md
docs/MEMORY_REGISTRY_SCOPE.md
docs/TEAM_EXECUTION_PLAN_V2.md
```
5. Summarize the new product definition, your directory ownership, the frozen TypeScript/API contract, required screens, demo path, prohibited old UI concepts, and acceptance tests.

Do not write code until you have confirmed the context reset.

YOUR ROLE:

You own the Memory Marketplace UI and the presentation-ready demo story.

YOU MAY MODIFY ONLY:

```
frontend/**
demo/start-memory-demo.mjs
demo/README_MEMORY_DEMO.md
```

YOUR REQUIRED SCREENS:

1. Explore Memories

Must show search, tag filters, featured memories, verified badge, author, version, stars, and installs.

2. Memory Detail

Must show title, author, version, problem, triggers, reusable steps, compatibility, verification evidence, fork lineage, and Install to Codex button.

3. Publish Memory

Must use a fixed validated form for a Memory Capsule. Do not provide raw Codex transcript upload.

4. My Codex

Must show installed memories, installed version, local installation status, and last used state.

5. Usage Receipt

Must clearly visualize:

Developer A → published memory → Developer B installed → Codex consumed it → pyproject.toml changed → pytest passed.

FIXED FEATURED MEMORY:

Title: Fix duplicate pytest module collisions

Slug: fix-pytest-module-collisions

ID: mem_pytest_importlib_v1

Problem: Pytest import file mismatch caused by duplicate test module basenames.

Method: Configure pytest with --import-mode=importlib.

Tags: python, pytest, debugging, codex

Status: verified

YOUR DEMO PATH:

Explore → search for pytest → open Fix duplicate pytest module collisions → click Install to Codex → show installed state in My Codex → run or replay the usage → show the verified Usage Receipt.

The product must be understandable within five seconds as “GitHub for Codex memories.”

DESIGN PRIORITIES:

- Large, clear information hierarchy.
- Search and Memory cards visible immediately.
- Verified evidence should be prominent.
- Install action should be obvious.
- Usage Receipt should be readable from a projector.
- Avoid dense dashboard layouts and tiny text.
- Avoid slow letter-by-letter animation.
- Use immediate, responsive transitions.
- Ensure all important Demo actions are available without hunting through menus.

LIVE/REPLAY REQUIREMENT:

Live and Replay must be visibly different. Replay must always show:

REPLAY — RECORDED EVIDENCE

Never present replay data as live execution.

DO NOT:

- Do not show Scout, Builder, Tester, or Reviewer as primary product concepts.
- Do not use Agent Roster as the homepage.
- Do not use the old Mission Control dashboard.
- Do not modify backend files.
- Do not invent response fields outside docs/MEMORY_REGISTRY_SCOPE.md.
- Do not claim hidden Codex memory mutation.
- Do not upload raw transcripts.
- Do not continue the old UI branch.

DEVELOPMENT METHOD:

Follow strict RED → GREEN → REFACTOR.

Before implementation, write failing tests for:

1. Searching pytest finds the fixed memory.
2. Opening a Memory shows its problem, steps, and verification.
3. Clicking Install changes the UI to installed state.
4. My Codex displays the installed memory.
5. Usage Receipt displays the publisher-to-verification chain.
6. Replay is unmistakably labeled.

REQUIRED VERIFICATION:

```
cd frontend
npm ci
npm test -- --run
npm run typecheck
npm run build
npm audit
```

Then start the real demo server, open it in a browser, verify every required screen at common projector dimensions, check for clipping/overlap/overflow/tiny text, check the browser console, and require zero JavaScript errors.

DELIVERY:

Push only to feat/memory-marketplace-ui.

Open a Draft PR targeting pivot/memory-registry-v1. The PR description must include screenshots of required screens, actual test output, production build output, npm audit result, browser-console result, live/replay status, and remaining blockers.

Begin by reading the required documents and reporting your understanding. Do not begin implementation until the branch and product direction are confirmed.

发送前检查

- 每个人只接收自己对应的一条完整指令。
- 确认成员本地切换到指令中指定的新分支。
- 确认 `git rev-parse HEAD` 对应统一基线，或该分支已基于该基线继续产生新提交。
- 要求 Codex 先报告上下文理解，再开始实现。
- 任何共享契约变更都必须先协调，不能由单个分支自行修改。
- 所有 Draft PR 的目标分支均为 `pivot/memory-registry-v1`。